

Curso prático de Git

Do Básico ao Avançado

Lambari

Equipe Imperador - UTFPR Curitiba

Julho 2026

Tópicos

- 1 Módulo 1: Básico (Github, repositório, etc.)
- 2 Módulo 2: Intermediário (Branches, Merges e .gitignore)
- 3 Módulo 3: Avançado (Revisão e Git Rebase)
- 4 Módulo 4: O Processo da Equipe (Implementação)

Módulo 1: Básico (Github, repositório, etc.)

Objetivo

Compreender a filosofia do Git, configurar o ambiente local e dominar o fluxo de trabalho essencial para arquivos individuais.

Tópicos abordados:

- O que é o Git vs. GitHub
- Configuração inicial do ambiente
- Ciclo de vida de um arquivo
- Comandos fundamentais locais e remotos

Git vs. GitHub

Git

- Sistema de controle de versão **local** e distribuído.
- Funciona inteiramente no terminal do computador.
- Tira "fotos" (commits) do histórico do código.

GitHub / GitLab

- Plataforma de hospedagem **na nuvem**.
- Interface gráfica para gerenciar repositórios.
- Adiciona ferramentas de colaboração (Pull Requests).

Configuração Inicial do Ambiente

Antes do primeiro commit, o Git precisa saber quem é você para registrar a autoria corretamente.

```
# Definir seu nome globalmente
$ git config --global user.name "Seu Nome Completo"

# Definir seu e-mail de trabalho
$ git config --global user.email "seu_email@imperador.com"

# Verificar configuracoes ativas
$ git config --list
```

O Ciclo de Vida dos Arquivos

No Git, os arquivos passam por estados específicos antes de serem consolidados:

Untracked Arquivo novo no diretório, o Git ainda não o monitora.

Modified Arquivo monitorado que sofreu alterações locais.

Staged Arquivo selecionado e pronto para fazer parte do próximo commit.

Committed Alterações salvas com segurança no banco de dados local.

Iniciando um Repositório: git init

O comando `git init` é utilizado para criar um novo repositório Git vazio ou reinicializar um já existente.

Atenção: Na prática do dia a dia, é mais comum criarmos o repositório diretamente em uma plataforma como o **GitHub** e depois clonarmos para a nossa máquina, tornando o `git init` um comando raramente utilizado manualmente pela nossa equipe.

```
# Inicializar um repositorio vazio na pasta atual  
$ git init  
# Clonar um repositorio da nuvem na past atual  
$ git clone <URL>
```

Verificando o Estado: git status

O comando `git status` exibe o estado do diretório de trabalho e da área de preparação (staging area).

Ele permite ver quais arquivos foram modificados, quais estão prontos para o próximo commit e quais ainda não estão sendo rastreados pelo Git. É o seu "radar" de desenvolvimento.

```
# Verificar o estado atual dos arquivos e modificacoes
$ git status
```

```
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   file1.txt
    new file:   file2.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    file3.txt
```


Preparando Arquivos: git add

O comando `git add` adiciona uma alteração do seu diretório de trabalho à área de preparação (staging area).

Isso avisa ao Git que você quer incluir as atualizações de um arquivo específico no próximo commit.

```
# Adicionar o script modificado para a area de preparacao  
$ git add sensor_velocidade.cpp  
# Adiciona todas as modifica es para a area de preparacao  
$ git add .
```

Salvando as Alterações: git commit

O comando `git commit` captura um "instantâneo" das alterações que estão atualmente na área de preparação (staging area).

Todo commit deve ser acompanhado de uma mensagem clara e descritiva para explicar o motivo da alteração no histórico do projeto.

```
# Salvar permanentemente o estado atual com uma breve ferramentas  
$ git commit -m "feat(sensores): adiciona leitura de pulsos do sensor Hall"  
# Salvar permanentemente o estado atual com uma mensagem mais detalhada em  
um editor de texto  
$ git commit
```

Trabalhando com o Repositório Remoto

Sincronização com o servidor em nuvem da equipe:

```
# Clonar um repositório existente para sua máquina local  
$ git clone <URL>  
  
# Enviar seus commits locais para a nuvem  
$ git push origin main  
  
# Buscar as novidades da nuvem e atualizar seu código local  
$ git pull origin main
```

Módulo 2: Intermediário (Branches, Merges e .gitignore)

Objetivo

Aprender a trabalhar em paralelo usando ramificações, integrar desenvolvimentos diferentes e gerenciar arquivos descartáveis.

Tópicos abordados:

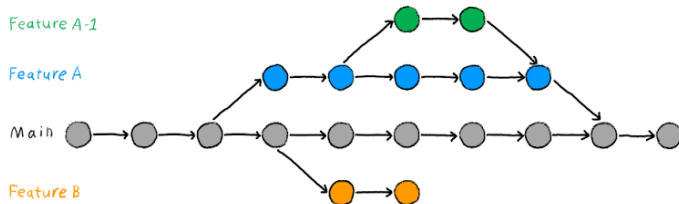
- Conceito e comandos de Branches
- Integração de código com Merge
- Resolução controlada de conflitos
- Configuração estratégica do .gitignore

O Propósito das Branches

Uma **branch** (ramificação) é uma linha de desenvolvimento independente. Funciona como um "universo paralelo" onde você pode alterar arquivos livremente sem afetar o código oficial (a branch `main`).

Por que usar?

- **Isolamento:** Evita que seu código em desenvolvimento quebre a versão principal.
- **Trabalho Paralelo:** Vários membros da equipe podem programar simultaneamente.
- **Organização:** Permite revisar a funcionalidade antes de integrá-la.



Ramificações (Branches)

Uma branch isola o seu desenvolvimento.

Regra na equipe: Nunca faça commits diretamente na branch main.

```
# Lista as branches que voce ja utilizou nesse repositorio
$ git branch

# Deletar uma branch do seu computador
$ git branch -D <nome da branch>

# Alterar para uma branch
$ git switch feature/filtros-sinais

# Comando alternativo (mais antigo) para alterar de branch
$ git checkout feature/filtros-sinais
```

Fusão de Código: Unindo Forças (Merge)

Como integrar o trabalho finalizado de volta à linha principal do projeto:

```
# 1. Voltar para a branch principal
$ git switch main

# 2. Atualizar a main local com o servidor
$ git pull origin main

# 3. Fundir branches (feito no PR do GitHub)
$ git merge feature/filtros-sinais

# 4. Remover a branch local que já foi integrada
$ git branch -d feature/filtros-sinais
```

Lidando com Conflitos de Frente

Conflitos ocorrem quando duas pessoas alteram a mesma linha do mesmo arquivo. O Git pausa o processo e exibe os marcadores:

```
<<<<<< HEAD
limite_rpm = 3800  # Codigo atual na Main
=====
limite_rpm = 4000  # Alteracao feita na sua Branch
>>>>>> feature/ajuste-motor
```

Resolução: Abra o arquivo, analise com a equipe qual lógica deve prevalecer, remova os marcadores (<<<, ==, >>>), salve, execute `git add` e finalize com `git commit`.

O Arquivo .gitignore

Evite que arquivos gerados por compilação, logs temporários ou pastas de IDEs sujem o histórico do repositório. Basta criar um arquivo chamado .gitignore na raiz com as extensões ou pastas mapeadas:

```
# Ignorar binarios e objetos de compilacao  
*.o  
*.bin  
*.elf  
build/  
.vscode/  
/logs/*.csv
```

Módulo 3: Avançado (Revisão e Git Rebase)

Objetivo

Dominar ferramentas avançadas para manipulação fina do histórico, limpeza de árvores de commits e recuperação de dados perdidos.

Tópicos abordados:

- Armazenamento temporário com `git stash`
- Organização linear com `git rebase`
- Importação cirúrgica com `git cherry-pick`
- Recuperação extrema com `git reflog`

Guardando Trabalho Incompleto: git stash

Se você estiver trabalhando em um código complexo e surgir um bug urgente na pista que exige troca imediata de branch:

```
# Salva o progresso inacabado em um compartimento isolado e limpa o terminal
$ git stash save -m "Progresso do desenvolvimento da telemetria"

# Agora voce esta livre para alternar para a main e corrigir o problema
$ git switch main
# ... correcao do bug ...

# De volta sua branch de trabalho, resgate as modificacoes salvas
$ git switch feature/telemetria
$ git stash pop
```

Histórico Linear: Rebase vs. Merge

Enquanto o merge cria um commit de junção explícito, o rebase reposiciona os commits da sua branch diretamente no topo da branch de destino, gerando um histórico limpo e sem ramificações visuais complexas.

```
# Atualiza sua branch aplicando suas modificacoes sobre a main atualizada
$ git switch feature/minha-feature
$ git rebase main
```

Aviso importante: Nunca utilize rebase em branches públicas/compartilhadas com outros membros da equipe para evitar dessincronização de históricos.

A Caixa-Preta do Git: git reflog

Se uma branch foi deletada acidentalmente ou um comando de reset removeu arquivos válidos, o reflog salva o dia registrando cada alteração local do ponteiro HEAD.

```
# Exibir o historico completo de operacoes locais
$ git reflog

# Saida esperada no terminal:
# e45a2d1 HEAD@{0}: reset: moving to HEAD~1
# a78b112 HEAD@{1}: commit: Adiciona calculo de G-Force

# Retornar o estado do projeto exatamente para o instante anterior ao erro:
$ git reset --hard a78b112
```

Módulo 4: O Processo da Equipe (Implementação)

Objetivo

Estabelecer a metodologia de trabalho da equipe. Padronizar a integração de código para garantir que o repositório sempre esteja funcional e validado.

Tópicos abordados:

- O modelo GitHub Flow adaptado ao ambiente competitivo
- Pull Requests (PRs) e Code Review
- Padronização com Commits Semânticos

Fluxo Operacional: GitHub Flow

Para a dinâmica veloz, adota-se o formato simplificado:

- 1 A branch `main` reflete o software funcional e testado no protótipo.
- 2 Desenvolvimentos ocorrem em branches nomeadas conforme o escopo (ex: `feature/sensor-freio`).
- 3 Commits são enviados para o servidor remoto.
- 4 A integração final exige a abertura de um **Pull Request** (PR).

Padronização de Branches (Parte 1)

Para manter a organização do repositório, adotamos uma convenção para as branches principais:

- `main`: Contém o código testado e 100% funcional, pronto para ir para o protótipo.
- `main/YYYY-MM-DD`: Branches de "snapshot". Salvam o estado exato do código em dias críticos (ex: competições), como `main/2026-07-05`.

Padronização de Branches (Parte 2)

As branches temporárias (de trabalho diário) seguem prefixos específicos que indicam o seu propósito:

- `feature/`: Utilizadas para desenvolvimentos de funções (ex: `feature/leitura-gps`).
- `bug/`: Utilizadas exclusivamente para correção de bugs no código (ex: `bug/falha-telemetria`).
- `test/`: Branches para desenvolver códigos voltados para testes (ex: `test/validacao-inversor`).
- `doc/`: Utilizadas exclusivamente para atualização de documentação em uma branch.

Segurança do Código: Pull Requests e Code Review

- **Branches Protegidas:** O envio direto de commits para a branch `main` deve ser bloqueado nas configurações do servidor remoto.
- **Revisão Técnica obrigatória:** Todo Pull Request precisa ser revisado e validado por pelo menos um outro integrante da área de eletrônica antes de ser integrado.
- **Impacto Direto:**
 - Previne que falhas de sintaxe cheguem ao carro antes dos testes de campo.
 - Garante que mais de um membro da equipe domine o funcionamento daquela seção do software.

Mensagens de Commit Padronizadas

Evite mensagens genéricas como "ajustes" ou "agora vai". Adote a convenção de Commits Semânticos:

```
# Estrutura padrao: tipo(subsistema): descricao em letras minusculas
$ git commit -m "feat(telemetria): adiciona transmissao via modulo LoRa"
$ git commit -m "fix/dashboard): corrige escala do grafico de RPM"
$ git commit -m "docs(readme): atualiza esquema de pinagem do
    microcontrolador"
$ git commit -m "test(pwt): otimizacao cvt"
```

Tipos recomendados:

- **feat**: Implementação de uma nova funcionalidade ou bloco de código.
- **fix**: Correção de uma falha ou bug operacional.
- **docs**: Alterações estritas na documentação técnica ou comentários.
- **test**: Código de um teste específico para um ou mais subsistemas.

Passo a Passo do Desenvolvimento

Fluxo de trabalho ideal do início ao fim de uma nova tarefa:

- 1 **Criar a branch no GitHub** (ex: feature/leitura-gps)
- 2 **Atualizar repositório local:** `git pull`
- 3 **Alternar para a branch:** `git switch feature/leitura-gps`
- 4 **Desenvolver:** Escrever o código, testar, etc.
- 5 **Atualizar branch (se a main mudou):** `git rebase main` (resolver eventuais conflitos)
- 6 **Preparar alterações:** `git add .` (ou arquivo específico)
- 7 **Salvar:** `git commit -m "feat(telemetria): rotina do GPS"`
- 8 **Enviar:** `git push origin feature/leitura-gps`
- 9 **Abrir o PR** (Pull Request) no GitHub.
- 10 **Realizar o Merge** (após aprovação no Code Review).